

Deep Learning Framework From Scratch in NumPy

Project Bible — Interview Preparation Guide

What this document is:

Everything you need to understand, explain, and defend
this project
in any interview. Written in plain English, no jargon.

Prakash Kumar Sarangi
MTech CS, NIT Calicut

CS6302E — Theoretical Foundations of Machine Learning

May 18, 2026

Contents

1	What Is This Project? (Plain English Overview)	3
1.1	The One-Line Summary	3
1.2	What Does “From Scratch” Actually Mean?	3
1.3	Project Journey	3
1.4	What the Project Produces	4
2	End-to-End Walk-Through — The Complete Story	4
2.1	The Full Story in Plain Prose	4
2.2	The Interview-Ready Version (Say This)	7
3	Project Structure — Every File Explained	8
3.1	<code>deep_neural_network.py</code> — The Heart of the Project	8
3.2	<code>gradient_check.py</code> — The Proof of Correctness	9
3.3	<code>train_mnist.py</code> — MNIST Training Script	9
3.4	<code>train_cifar10.py</code> — CIFAR-10 Benchmark	10
3.5	<code>architecture_search_mnist.py</code> — Finding the Best Size	10
3.6	<code>app.py</code> — The Live Demo	10
3.7	<code>mnist_loader.py</code> and <code>cifar10_loader.py</code> — Data Pipelines	10
4	Core Concepts — Built From the Ground Up	10
4.1	What Is a Neural Network?	10
4.2	Forward Pass — How Input Becomes Output	11
4.3	Activation Functions	11
4.3.1	ReLU (Rectified Linear Unit) — Used in Hidden Layers	12
4.3.2	Sigmoid	12
4.3.3	Tanh	12
4.3.4	Softmax — Used in Output Layer	12
4.4	Loss Functions — Measuring How Wrong We Are	12
4.4.1	MSE (Mean Squared Error)	13
4.4.2	Cross-Entropy Loss — The Right Choice for Classification	13
4.5	Backpropagation — How the Network Learns	13
4.6	Optimizers — How We Update the Weights	14
4.6.1	SGD (Stochastic Gradient Descent)	14
4.6.2	Adam (Adaptive Moment Estimation) — What This Project Uses	15
4.7	Weight Initialisation — Why It Matters	15
4.7.1	He Initialisation — Used for ReLU Layers	15
4.7.2	Xavier Initialisation — Used for Sigmoid/Tanh	16
4.8	Mini-Batch Training	16
4.9	L2 Regularisation	16
4.10	Gradient Checking — Proving Backprop is Correct	16
4.11	The MNIST Dataset	17
4.12	The CIFAR-10 Dataset	17
4.13	Architecture Search	18
5	The Interview Script	19
5.1	The 30-Second Version (Elevator Pitch)	19

5.2	The 2-Minute Version (Standard Interview Answer)	19
5.3	Explaining the MSE vs Cross-Entropy Insight	19
5.4	Explaining the CIFAR-10 Result	20
6	Interview Questions and Answers	20
6.1	Questions About Backpropagation	20
6.2	Questions About the Optimiser	21
6.3	Questions About the Results	22
6.4	Questions About the Framework Design	22
6.5	General ML Questions This Project Prepares You For	23
7	Key Numbers to Memorise	24
8	What Makes This Project Stand Out	24
9	What You Should NOT Claim	24
10	Quick Revision — The 10-Point Cheat Sheet	25

What Is This Project? (Plain English Overview)

The One-Line Summary

“I built a neural network training library from scratch — like a mini version of PyTorch — using only NumPy, and tested it on two real image datasets.”

What Does “From Scratch” Actually Mean?

When most people do ML projects, they write:

```
1 model = Sequential([Dense(128, activation='relu'), Dense(10,
    activation='softmax')])
2 model.compile(optimizer='adam', loss='categorical_crossentropy',
    )
3 model.fit(X_train, y_train, epochs=30)
```

The framework (Keras/PyTorch) does all the actual work. You don't know what's happening inside.

This project is different. Every single mathematical operation is implemented by hand:

- The forward pass (how inputs become predictions)
- Backpropagation (how the network learns from its mistakes)
- The Adam optimizer (a smarter version of gradient descent)
- Weight initialization strategies
- Numerical gradient verification (a mathematical test that confirms the math is correct)

Project Journey

Start	→	Extension	→	Final
Course assignment: classify letters B, 0, E on tiny 8x8 images		Port the same code to MNIST (60,000 real handwritten digits)		Build a full reusable framework: N lay- ers, Adam, CE loss, CIFAR-10

What the Project Produces

Benchmark	Architecture	Accuracy
MNIST (digits 0–9)	784 – 128 – 10	97.66%
CIFAR-10 (10 image classes)	3072 – 512 – 256 – 10	50.95%
Random guess	—	10.00%
CNN (for comparison)	ResNet etc.	~93%

Note on CIFAR-10 51%: This is not a failure. It is the point of the experiment. The project deliberately uses an MLP (fully connected network) on CIFAR-10 to show *why* MLPs cannot compete with CNNs on image data — and explains the exact reason.

End-to-End Walk-Through — The Complete Story

This is the section to memorise. If an interviewer says “walk me through your project from start to finish,” this is exactly what you say. It is a single, connected story — not bullet points, not definitions. Read it 5 times, then close the document and say it aloud.

The Full Story in Plain Prose

The goal: Train a neural network to look at a handwritten digit image and say which digit (0–9) it is.

Step 1 — Raw Data

We start with the MNIST dataset: 70,000 greyscale images of handwritten digits, each 28×28 pixels. We split them into 60,000 for training and 10,000 for testing. The test set is locked away and never used during training — it is the final exam.

Each image is a grid of pixel values between 0 (black) and 255 (white). Before feeding anything into the network, we **normalise** the pixel values to the range $[-1, +1]$ using the formula $X = X/127.5 - 1.0$. This is important because large input values cause exploding pre-activations, and our weight initialisation assumes inputs are centred around zero.

We also **one-hot encode** the labels. The digit “3” becomes the vector $[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]$. This is needed because the network’s output is a 10-dimensional probability vector, and the loss function needs the target in the same shape.

Step 2 — Network Initialisation

We create a `DeepNeuralNetwork([784, 128, 10])`. This means:

- Input layer: 784 neurons (one per pixel)
- Hidden layer: 128 neurons
- Output layer: 10 neurons (one per digit class)

The network needs two weight matrices and two bias vectors:

- W_1 : shape (784, 128) — connects input to hidden layer
- b_1 : shape (1, 128) — bias for hidden layer, initialised to zeros
- W_2 : shape (128, 10) — connects hidden to output layer
- b_2 : shape (1, 10) — bias for output layer, initialised to zeros

We initialise W_1 and W_2 using **He initialisation**: random values drawn from a normal distribution with standard deviation $\sqrt{2/\text{fan_in}}$. The factor of 2 compensates for ReLU zeroing out roughly half of its inputs. Bad initialisation (all zeros, or too large) would prevent learning entirely.

Step 3 — Forward Pass (One Mini-Batch)

Training happens in mini-batches of 256 images. For one such batch, the forward pass computes:

1. $z_1 = X_{\text{batch}} \cdot W_1 + b_1$ — multiply input (256×784) by weights (784×128), add bias. Result: 256×128 .
2. $a_1 = \text{ReLU}(z_1)$ — apply ReLU element-wise. Any negative value becomes 0.
3. $z_2 = a_1 \cdot W_2 + b_2$ — multiply (256×128) by (128×10). Result: 256×10 .
4. $a_2 = \text{Softmax}(z_2)$ — convert 10 raw scores into 10 probabilities that sum to 1.

We save X_{batch} , a_1 , and a_2 in a **cache list**. We will need them in the backward pass.

Step 4 — Compute the Loss

We now have a_2 , a matrix of shape (256, 10), where each row is the network's probability distribution over 10 digits. We compare this against the true one-hot labels y using **cross-entropy loss**:

$$L = -\frac{1}{256} \sum_{i=1}^{256} \log(a_2[i, \text{true digit of image } i]) \quad (1)$$

In plain English: for each image, look at the probability the network assigned to the correct digit, take the log, negate it, and average over the batch. If the network said 99% on the right digit, the loss contribution is $-\log(0.99) \approx 0.01$ (small, good). If it said 1%, the contribution is $-\log(0.01) = 4.6$ (large, bad).

Step 5 — Backward Pass (Backpropagation)

This is where the network learns. We use the chain rule to compute how much each weight contributed to the loss, then adjust it.

Output layer: The gradient at the output is the cleanest part. For cross-entropy with softmax, it simplifies to:

$$\delta_2 = (a_2 - y) / 256 \quad (2)$$

This is just “predicted minus true, divided by batch size.” If the network said [0.1, 0.1, 0.7, 0.1...] but the true digit was 0, δ_2 is [0.1-1, 0.1-0, 0.7-0...] / 256 = [-0.9, 0.1, 0.7...] / 256. Large negative for the right class (we need to push it up), positive for wrong classes (push them down).

Output weight gradients:

$$\nabla W_2 = a_1^T \cdot \delta_2, \quad \nabla b_2 = \text{sum}(\delta_2, \text{axis} = 0) \quad (3)$$

Propagate error to hidden layer (the chain rule in action):

$$\delta_1 = (\delta_2 \cdot W_2^T) \odot \text{ReLU}'(a_1) \quad (4)$$

We ask: “which hidden neurons were responsible for this output error?” We multiply δ_2 by the transpose of W_2 to push the error signal backwards. Then we multiply by the ReLU derivative: neurons that were outputting a positive value (ReLU was open) pass the signal through; neurons that were outputting zero (ReLU was closed) block the signal entirely.

Hidden weight gradients:

$$\nabla W_1 = X_{\text{batch}}^T \cdot \delta_1, \quad \nabla b_1 = \text{sum}(\delta_1, \text{axis} = 0) \quad (5)$$

Step 6 — Update Weights (Adam Optimiser)

We have gradients for all four parameters (W_1, b_1, W_2, b_2). We use Adam to update them: For each parameter (e.g., W_1):

1. Update the **momentum**: $m \leftarrow 0.9 \cdot m + 0.1 \cdot \nabla W_1$ (smooth average of recent gradients)
2. Update the **velocity**: $v \leftarrow 0.999 \cdot v + 0.001 \cdot (\nabla W_1)^2$ (tracks gradient variance)
3. Apply **bias correction**: $\hat{m} = m / (1 - 0.9^t)$, $\hat{v} = v / (1 - 0.999^t)$
4. Update the weight: $W_1 \leftarrow W_1 - 0.001 \cdot \hat{m} / (\sqrt{\hat{v}} + 10^{-8})$

The key idea: if a weight has been consistently getting a large gradient in one direction, Adam takes a bigger step. If the gradient has been noisy and fluctuating, Adam takes a smaller, more cautious step.

Step 7 — One Epoch

Steps 3–6 are repeated for every mini-batch in the training set. With 60,000 training images and a batch size of 256, that is about 235 mini-batches per epoch. After all batches are processed, we have completed one epoch. We measure the accuracy on the full test set and record it.

Step 8 — 30 Epochs of Training

We repeat Step 7 thirty times. Each epoch, the network sees all 60,000 training images (in random order), and the weights are updated $235 \times 30 = 7,050$ times. The loss steadily decreases and accuracy climbs:

Epoch	Loss	Train Acc	Test Acc
1	0.499	92.6%	92.1%
5	0.128	97.2%	96.4%
15	0.050	99.0%	97.7%
30	0.017	99.8%	97.7%

Final result: 97.66% test accuracy — the network correctly classifies 9,766 out of 10,000 digits it has never seen before.

Step 9 — Verification (Gradient Check)

Separately, we run `gradient_check.py` to prove the backpropagation code is mathematically correct. For a tiny test network, we compute the gradient two ways:

- Analytically via backprop (our implementation)
- Numerically: perturb each weight by $\varepsilon = 10^{-5}$, measure change in loss, divide by 2ε

The relative error between the two is $\approx 10^{-10}$ — essentially machine precision. This is not a test we had to run; it is a proof that the math inside the framework is correct.

Step 10 — CIFAR-10 Experiment

We run the same framework on CIFAR-10: 50,000 colour images ($32 \times 32 \times 3 = 3,072$ pixels) across 10 classes. We use a larger network (3072–512–256–10) and train for 40 epochs. The result is **50.95% test accuracy** while training accuracy reaches 95.38% — the network memorises the training set but cannot generalise. Notably, vehicle classes do far better (ship 68.6%, automobile 61.4%) than animal classes (bird 36.1%, cat 36.6%), because animals have far more pose, texture, and viewpoint variation that a pixel-level model cannot capture.

This demonstrates a fundamental architectural limitation: an MLP flattens the image to a 1D vector, losing all spatial information. Pixel (5,5) and pixel (5,6) are just two independent numbers. The network can never learn that they are adjacent. A CNN, which processes local patches of pixels together, reaches 93%+ because it understands spatial structure. This experiment makes that gap concrete and measurable.

Step 11 — The Web App

We load the saved MNIST model weights into a Flask web application. The user opens a browser, draws a digit on an HTML canvas, and the app:

1. Captures the canvas as a PNG image
2. Preprocesses it: convert to greyscale, invert (canvas is white-on-black, MNIST is black-on-white), crop to bounding box, pad 20%, resize to 28×28 , normalise to $[-1, +1]$
3. Flattens to a 784-dim vector and feeds it to the trained network
4. Returns the predicted digit and confidence scores for all 10 classes

The Interview-Ready Version (Say This)

“Sure. The goal was to build a neural network from scratch in NumPy — meaning I implemented every operation by hand, no PyTorch or TensorFlow. I start with the MNIST dataset: 70,000 handwritten digit images. I normalise the pixel values to $[-1, +1]$ and one-hot encode the labels. Then I initialise a three-layer network: 784 inputs, 128 hidden neurons, 10 outputs. The weights are set using He

initialisation, which ensures the signal doesn't explode or vanish in the first forward pass.

Training runs in mini-batches of 256 images. For each batch: I do a forward pass — matrix multiply the input with the first weight matrix, apply ReLU, multiply again, apply Softmax to get probabilities. I cache every layer's output because I'll need them in the backward pass.

Then I compute the cross-entropy loss — essentially, how surprised was the network by the correct label?

Then backpropagation. At the output layer, the gradient simplifies cleanly to 'predicted minus true, divided by batch size'. I use the chain rule to propagate this error signal backwards: multiply by the transpose of the weight matrix, then element-wise multiply by the ReLU derivative. This gives me the gradient for each layer's weights.

I update the weights using Adam: it keeps a running average of past gradients and a running average of squared gradients, using those to adapt the step size per parameter with bias correction.

After 30 epochs and about 7,000 weight updates, the network reaches 97.66% accuracy on 10,000 test images it has never seen.

I also ran the same framework on CIFAR-10 to demonstrate the MLP ceiling — it plateaus at 51%, which I use to explain why CNNs exist. And I built a Flask web app where you can draw a digit and get a live prediction."

Project Structure — Every File Explained

```
project/
  deep_neural_network.py      <-- The CORE of everything. The framework.
  gradient_check.py          <-- Mathematical proof that backprop is correct.
  train_mnist.py             <-- Train on MNIST, generate plots.
  train_cifar10.py           <-- Train on CIFAR-10, benchmark MLP ceiling.
  architecture_search_mnist.py <-- Find the best network size.
  app.py                     <-- Flask web app: draw a digit, get prediction.
  mnist_loader.py            <-- Download and prepare MNIST data.
  cifar10_loader.py          <-- Download and prepare CIFAR-10 data.
  neural_network.py          <-- Original assignment code (kept for history).
  templates/index.html       <-- Web UI for the Flask app.
```

deep_neural_network.py — The Heart of the Project

This is the main deliverable. Everything else just calls this file. Think of it as your own mini-PyTorch.

What it supports:

- **Any number of layers:** You pass a list like [784, 256, 128, 64, 10] and it builds a 4-layer network automatically.

- **Three activations:** ReLU, Sigmoid, Tanh
- **Two losses:** Cross-Entropy (with Softmax output), MSE (with Sigmoid output)
- **Two optimizers:** Adam, SGD
- **Weight init:** He initialization (for ReLU), Xavier (for Sigmoid/Tanh)
- **L2 regularization:** Prevents overfitting
- **Save/Load:** Stores trained weights to a `.npz` file and loads them back

How you use it (3 lines):

```
1 nn = DeepNeuralNetwork([784, 128, 10], activation='relu', loss='cross_entropy', optimizer='adam')
2 nn.train(X_train, y_train, epochs=30, X_val=X_test, y_val=y_test)
3 accuracy = nn.get_accuracy(X_test, y_test)
```

gradient_check.py — The Proof of Correctness

This script answers the question: “How do you know your backpropagation code is correct?”

It uses a technique called **numerical gradient checking**: it computes the gradient two ways — mathematically (via backprop) and numerically (by tweaking each weight slightly and measuring the change in loss). If both methods agree to within 10^{-10} error, the code is verified correct.

Result: All 3 test configurations (ReLU+CE, Sigmoid+MSE, Tanh+CE) pass.

train_mnist.py — MNIST Training Script

What it does:

1. Downloads MNIST (60,000 digit images) on first run
2. Creates a `DeepNeuralNetwork([784, 128, 10])`
3. Trains for 30 epochs with Adam, batch size 256
4. Saves the trained model to `model/mnist_dnn_model.npz`
5. Generates 4 result plots in `results/`

Plots generated:

- `mnist_training_curves.png` — Loss and accuracy per epoch
- `mnist_weight_maps.png` — 128 hidden neurons visualized as 28x28 images
- `mnist_confusion_matrix.png` — Which digits get confused with which
- `mnist_sample_predictions.png` — Correct and wrong predictions shown

`train_cifar10.py` — CIFAR-10 Benchmark

What it does: Same as above but for CIFAR-10 (50,000 colour images, 10 classes). Uses a bigger network: 3072–512–256–10. Trains for 40 epochs.

Purpose: Not to get high accuracy, but to demonstrate the MLP ceiling (50.95%) and explain why CNNs (which use local filters) reach 93%.

`architecture_search_mnist.py` — Finding the Best Size

Tests 6 different hidden layer sizes: 16, 32, 64, 128, 256, 512. Runs 2 trials each. Reports mean and standard deviation of test accuracy. This is how you do systematic experiment design — something interviewers notice.

`app.py` — The Live Demo

A Flask web application. You open a browser, draw a digit on a canvas, and the trained MNIST network classifies it instantly. Shows the top predicted digit and a confidence bar for all 10 digits.

The tricky part: The canvas is white-background with black strokes. MNIST is black-background with white digits. The preprocessing pipeline inverts the image to match MNIST’s convention before passing it to the network.

`mnist_loader.py` and `cifar10_loader.py` — Data Pipelines

These handle downloading, caching, normalizing, and one-hot encoding the datasets.

MNIST normalization: $X = X/127.5 - 1.0$ (pixels go from [0,255] to [-1, +1])

CIFAR-10 normalization: Per-channel zero-mean, unit-variance using training set statistics. Each of the 3 colour channels (R, G, B) is normalized separately.

Core Concepts — Built From the Ground Up

This section explains every concept used in the project in plain English. Read this once and you will be able to answer questions without memorizing definitions.

What Is a Neural Network?

A neural network is a **mathematical function** that takes in numbers (e.g., pixel values of an image), performs a series of matrix multiplications and non-linear transformations, and outputs probabilities (e.g., “this image is 92% likely to be the digit 3”).

It “learns” by adjusting its internal numbers (called **weights**) based on how wrong its predictions are.

The structure:

- **Input layer:** The raw data. For MNIST, this is 784 numbers (28x28 pixels flattened).
- **Hidden layers:** Intermediate transformations. Our MNIST network has one hidden layer of 128 neurons.
- **Output layer:** The final predictions. For MNIST (10 digits), this is 10 numbers representing probabilities.

What is a “neuron”? Just a number. Specifically, the output of a weighted sum passed through an activation function.

Forward Pass — How Input Becomes Output

The forward pass is the calculation that takes your input and produces a prediction.

For our 784–128–10 MNIST network:

$$z_1 = X \cdot W_1 + b_1 \quad (\text{linear transformation, shape: } n \times 128) \quad (6)$$

$$a_1 = \text{ReLU}(z_1) \quad (\text{apply activation, shape: } n \times 128) \quad (7)$$

$$z_2 = a_1 \cdot W_2 + b_2 \quad (\text{linear transformation, shape: } n \times 10) \quad (8)$$

$$a_2 = \text{Softmax}(z_2) \quad (\text{probabilities that sum to 1}) \quad (9)$$

Where:

- X is the input (shape: $n \times 784$, where n = batch size)
- W_1 is the weight matrix connecting input to hidden layer (shape: 784×128)
- b_1 is the bias vector (shape: 1×128)
- W_2, b_2 are weights and biases for the output layer

In plain English: Each layer takes the output of the previous layer, multiplies it by a matrix of learned weights, adds a bias, and squashes the result through a non-linear function.

The weight matrix W is where the network “stores” what it has learned. Each element W_{ij} represents how strongly input feature i influences neuron j . Training is just adjusting these numbers.

Activation Functions

Why do we need them? Without activation functions, stacking multiple linear layers is mathematically equivalent to a single linear layer. No amount of depth would help. Activation functions introduce non-linearity, allowing the network to learn complex patterns.

ReLU (Rectified Linear Unit) — Used in Hidden Layers

$$\text{ReLU}(z) = \max(0, z) \quad (10)$$

If the input is positive, pass it through unchanged. If negative, output zero.

Why ReLU?

- **Fast:** Just a comparison operation. No exponentials.
- **Sparse:** Many neurons output exactly 0, making the network efficient.
- **No vanishing gradient problem:** Gradient is 1 for positive inputs (doesn't shrink during backprop).

Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (11)$$

Squashes any number to the range (0, 1). Useful for binary probabilities.

Problem: For very large or very small z , the gradient is nearly 0. This is the **vanishing gradient problem** — learning slows to almost nothing in deep networks.

Tanh

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad (12)$$

Squashes to (-1, 1). Better than sigmoid because it's zero-centred (outputs average to 0), but still suffers from vanishing gradients.

Softmax — Used in Output Layer

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (13)$$

Converts a vector of arbitrary numbers into a probability distribution (all positive, sum to 1).

Example: If the output layer gives [2.1, 0.5, 3.3], softmax converts this to [0.20, 0.09, 0.71]. The network is 71% confident this is class 3.

We subtract the maximum value before computing softmax: $z_i \leftarrow z_i - \max(z)$. This prevents overflow (very large exponentials). The result is mathematically identical but numerically stable.

Loss Functions — Measuring How Wrong We Are

A loss function takes the network's prediction and the true label, and returns a single number representing the error. Training tries to minimize this number.

MSE (Mean Squared Error)

$$L_{MSE} = \frac{1}{n} \sum_{i=1}^n \sum_j (y_{ij} - \hat{y}_{ij})^2 \quad (14)$$

The average of the squared differences between predicted and true values.

Problem for classification: MSE treats all errors as equally bad in a symmetric way. For classification (where probabilities should be sharp), it gives weak gradients when the network is already close to the right answer, causing slow learning.

Cross-Entropy Loss — The Right Choice for Classification

$$L_{CE} = -\frac{1}{n} \sum_{i=1}^n \sum_j y_{ij} \cdot \log(\hat{y}_{ij}) \quad (15)$$

Since y is one-hot (only one class is 1, rest are 0), this simplifies to:

$$L_{CE} = -\frac{1}{n} \sum_{i=1}^n \log(\hat{y}_{i, \text{true class}}) \quad (16)$$

In plain English: “How surprised was the network by the correct answer?” If the network said 99% probability on the right class, $-\log(0.99) \approx 0.01$ — low loss, good. If it said 1% on the right class, $-\log(0.01) = 4.6$ — high loss, bad.

This project achieves **97.66%** with Cross-Entropy + Softmax.
The original code used MSE + Sigmoid and achieved only **92%**.
Why? Cross-entropy combined with Softmax produces a beautifully clean gradient:

$$\frac{\partial L_{CE}}{\partial z_{\text{output}}} = \hat{y} - y \quad (17)$$

This is just “predicted minus true.” Strong, clean signal that drives fast, stable learning. MSE with sigmoid produces a weaker, more muddled gradient.

This is the single most important concept in this project to understand and explain.

Backpropagation — How the Network Learns

During training, the network makes a prediction, we measure how wrong it was (the loss), and then we need to figure out: “which weights caused this mistake, and by how much?” Backpropagation is the algorithm that answers this question using the chain rule of calculus.

The chain rule in English: If C depends on B and B depends on A , then the rate at which C changes with respect to A is: (how fast C changes with B) $\times$ (how fast B changes with A).

For our 2-layer network:

Step 1: Output layer error (the “delta”)

For CE + Softmax, the combined gradient is:

$$\delta_2 = \frac{\hat{y} - y}{n} \quad (18)$$

Step 2: Output layer weight gradients

$$\nabla W_2 = a_1^T \cdot \delta_2, \quad \nabla b_2 = \sum \delta_2 \quad (19)$$

Step 3: Propagate error back to hidden layer

$$\delta_1 = (\delta_2 \cdot W_2^T) \odot \text{ReLU}'(a_1) \quad (20)$$

Where $\odot$ means element-wise multiplication and $\text{ReLU}'(a_1) = \mathbf{1}[a_1 > 0]$ (1 if positive, 0 otherwise).

Step 4: Hidden layer weight gradients

$$\nabla W_1 = X^T \cdot \delta_1, \quad \nabla b_1 = \sum \delta_1 \quad (21)$$

Step 5: Update weights (SGD rule)

$$W \leftarrow W - \alpha \cdot \nabla W \quad (22)$$

Where α is the learning rate.

The generalisation to N layers: The pattern repeats. You start from the output, compute the delta, compute the gradients for that layer, propagate the delta one step back using the weight matrix and activation derivative, and repeat until you reach the input. This is implemented as a loop in `_compute_grads()` in `deep_neural_network.py`.

Optimizers — How We Update the Weights

SGD (Stochastic Gradient Descent)

The simplest update rule:

$$W \leftarrow W - \alpha \cdot \nabla W \quad (23)$$

Problem: You need to carefully tune the learning rate α . If too large: weights oscillate and training diverges. If too small: training is very slow.

Adam (Adaptive Moment Estimation) — What This Project Uses

Adam is smarter. It keeps a running memory of past gradients and automatically adjusts the learning rate for each weight individually.

The four update equations:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{1st moment: exponential average of gradients}) \quad (24)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{2nd moment: exponential average of squared gradients}) \quad (25)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (\text{bias correction}) \quad (26)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{bias correction}) \quad (27)$$

$$W \leftarrow W - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \quad (\text{weight update}) \quad (28)$$

Parameters used: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, $\alpha = 0.001$

m_t (momentum): A smoothed version of the current gradient, weighted toward recent history. If the gradient has been pointing in the same direction for many steps, m_t is large — take a bigger step.

v_t (adaptive learning rate): Tracks how much the gradient has been fluctuating. If a weight's gradient fluctuates wildly, v_t is large and we divide by it — take a smaller step (be cautious).

Bias correction: At the start of training, m_t and v_t are initialized to 0. Without correction, early updates are too small. Dividing by $(1 - \beta^t)$ compensates.

Result: Adam works well with almost any learning rate, converges fast, and requires little tuning. This is why it became the default optimizer in the field.

Weight Initialisation — Why It Matters

Bad initialisation kills training.

- All zeros: Every neuron computes the same thing, learns the same thing. Network never learns.
- Too large: Activations explode, gradients explode, training diverges.
- Too small: Activations and gradients vanish, network learns nothing.

He Initialisation — Used for ReLU Layers

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{\text{fan.in}}}\right) \quad (29)$$

Where `fan.in` is the number of inputs to the layer. The factor of 2 compensates for ReLU setting roughly half of its inputs to zero.

Xavier Initialisation — Used for Sigmoid/Tanh

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{\text{fan_in} + \text{fan_out}}}\right) \quad (30)$$

Designed so that the variance of the output is equal to the variance of the input, which keeps gradients stable in both forward and backward passes.

Mini-Batch Training

Instead of updating weights after seeing all 60,000 images (full-batch) or after each single image (stochastic), we update after each small batch (e.g., 256 images).

Why?

- **Speed:** GPU/vectorization works best on batches, not single samples.
- **Noise:** Each batch is a slightly different random sample, which adds noise to the gradient. This noise actually helps escape local minima.
- **Memory:** You don't need all 60,000 images in memory at once.

Each epoch: Shuffle the training data, break into batches of 256, do a forward + backward pass on each batch, update weights after each batch.

L2 Regularisation

The problem: Overfitting. The network memorises the training data but performs poorly on new data.

The fix: Add a penalty term to the loss that punishes large weights:

$$L_{\text{total}} = L_{\text{data}} + \frac{\lambda}{2} \sum_l \|W_l\|^2 \quad (31)$$

Effect on gradient:

$$\nabla W_l \leftarrow \nabla W_l + \frac{\lambda}{n} W_l \quad (32)$$

The weight is “pulled towards zero” at each step. Large weights are penalised, which forces the network to spread the learning across many weights rather than relying too heavily on any single one.

Gradient Checking — Proving Backprop is Correct

There are two ways to compute the gradient of the loss with respect to a weight w :

1. **Analytically (backpropagation):** Fast, but you might have a bug.
2. **Numerically (finite differences):** Slow, but provably correct.

If both give the same answer, your backpropagation code is correct.

The numerical gradient formula (centered finite differences):

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \varepsilon) - L(w - \varepsilon)}{2\varepsilon} \quad (33)$$

For each weight, we:

1. Slightly increase it by $\varepsilon = 10^{-5}$ and measure the loss
2. Slightly decrease it by ε and measure the loss
3. Compute the slope (finite difference)
4. Compare with the analytical gradient from backprop

The relative error formula:

$$\text{relative error} = \frac{\|\nabla_{\text{analytical}} - \nabla_{\text{numerical}}\|}{\|\nabla_{\text{analytical}}\| + \|\nabla_{\text{numerical}}\|} \quad (34)$$

Threshold: $< 10^{-4}$ is considered correct. Our implementation achieves $\approx 10^{-10}$ (essentially machine precision).

The MNIST Dataset

- 70,000 greyscale images of handwritten digits (0–9)
- 60,000 training, 10,000 test
- Each image is 28x28 pixels = 784 values per image
- Standard benchmark used since 1998; still widely used today
- Normalised to $[-1, +1]$: $X = X/127.5 - 1.0$
- Labels: one-hot encoded (e.g., digit 3 = $[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]$)

The CIFAR-10 Dataset

- 60,000 colour images, 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck
- 50,000 training, 10,000 test
- Each image is 32x32 pixels $\times$ 3 channels (RGB) = 3,072 values per image
- Much harder than MNIST because of colour, texture, and viewpoint variation
- Normalised per channel (R, G, B separately): zero mean, unit variance

Why only 51% on CIFAR-10?

An MLP flattens the 32x32x3 image into a single vector of 3,072 numbers. It has no way of knowing that pixel (5,5) is adjacent to pixel (5,6). It treats every pixel as independent.

A CNN uses **local filters** (e.g., 3x3 kernels) that slide across the image, detecting edges, curves, and textures at any location. This spatial awareness is what allows CNNs to reach 93%+.

The lesson: Architecture choice matters. Not just layer sizes, but the fundamental structure (MLP vs CNN vs Transformer) determines what a model can learn.

Architecture Search

Question: How do you choose the hidden layer size (16? 128? 512)?

Answer: Run experiments. Test multiple sizes on the same dataset, measure test accuracy, pick the best.

Key principle: Always evaluate on a **held-out test set** that the model has never seen during training. Do not measure accuracy on the training set — that will always be high (the model memorises it) and tells you nothing about generalisation.

This project tests hidden sizes 16, 32, 64, 128, 256, 512 (2 trials each, mean $\pm$ std on the held-out test set):

Hidden size	Parameters	Mean test acc	Best
16	12,730	93.07% $\pm$ 0.70%	93.77%
32	25,450	94.96% $\pm$ 0.05%	95.01%
64	50,890	96.84% $\pm$ 0.04%	96.87%
128	101,770	97.69% $\pm$ 0.02%	97.72%
256	203,530	97.94% $\pm$ 0.11%	98.04%
512	407,050	97.60% $\pm$ 0.11%	97.71%

Observations (this is the story to tell):

- Very small networks (16 neurons) underfit — only 93%, not enough capacity to learn all digit variations.
- Accuracy rises steadily with capacity up to 256 neurons (the best: 97.94%).
- At 512 neurons accuracy actually *drops* slightly (97.60%) — too many parameters for the dataset, the network starts overfitting. More capacity is not always better.
- The standard deviation is tiny (± 0.02 – 0.11% for larger nets), confirming the results are stable and not luck.

Conclusion: The best architecture is 784--256--10 at 97.94%, but 128 is nearly as good (97.69%) with half the parameters — a sensible accuracy/size trade-off.

The Interview Script

These are scripts, not essays. Practise saying them out loud. The goal is to be able to explain the project in 30 seconds or 2 minutes without thinking.

The 30-Second Version (Elevator Pitch)

“I built a deep learning framework from scratch in NumPy — no PyTorch, no TensorFlow, nothing. The framework supports networks of any depth with multiple activation functions, uses the Adam optimiser with bias correction, and I verified the backpropagation is mathematically correct using numerical gradient checking. I tested it on MNIST where it achieves 97.66% accuracy, and on CIFAR-10 where it intentionally demonstrates the 51% ceiling of MLPs compared to CNNs.”

The 2-Minute Version (Standard Interview Answer)

“This project started as a course assignment where I had to classify 3 letters on tiny synthetic images. After completing that, I decided to go much deeper and turn it into a proper mini deep learning framework.

The core of the project is a class called `DeepNeuralNetwork` that you configure like a PyTorch module — you pass in the layer sizes, pick your activation function, loss function, and optimiser, and it builds and trains the network for you. Under the hood, I implemented the forward pass with full activation caching, N-layer generalised backpropagation using the chain rule, and the Adam optimiser with both momentum and adaptive learning rates.

One thing I’m especially proud of is the gradient check. A lot of people implement backprop and just hope it’s right. I wrote a separate script that verifies it numerically — by perturbing each weight slightly and comparing the analytical gradient with the finite difference approximation. My implementation achieves 10^{-10} relative error, which is essentially machine precision.

For results: on MNIST, I get 97.66% test accuracy. For context, the original assignment code using MSE loss only reached 92% — switching to cross-entropy with softmax was the key insight that drove that improvement. I also trained a bigger network on CIFAR-10, which plateaus at about 51%. That’s intentional — it demonstrates that MLPs can’t do better than that because they’re blind to spatial structure in images, which is exactly what CNNs fix.

I also built a Flask web app where you can draw a digit on a canvas and the network classifies it in real time.”

Explaining the MSE vs Cross-Entropy Insight

This is likely to come up. Here is the exact way to explain it:

“The original code used MSE loss — squared difference between predicted and true values. When I switched to cross-entropy with softmax, accuracy jumped from 92% to 97.66%.

The reason is the gradient signal. With CE+Softmax, the gradient at the output layer simplifies to just ‘predicted minus true’, divided by batch size. Clean, strong signal. With MSE+sigmoid, you get an extra sigmoid-derivative term multiplied in, which is very small when the network is already outputting values near 0 or 1 — this is the vanishing gradient problem. The network gets stuck because the gradient is too weak to move the weights meaningfully.

Cross-entropy is the correct loss for classification. MSE is for regression problems where the output is a continuous value.”

Explaining the CIFAR-10 Result

“My network gets 51% on CIFAR-10, and that’s not a failure — it’s a demonstration. An MLP flattens the image into a vector of 3,072 numbers. Pixel at position (5,5) and pixel at (5,6) are just two independent numbers to the MLP; it has no idea they’re adjacent. So it can’t detect that there’s an edge between them, or that there’s a circular shape, or anything spatial.

A CNN uses local filters — a 3x3 patch of pixels is processed together, and the same filter slides across the whole image. This gives CNNs translational invariance and the ability to detect edges, textures, and shapes regardless of where they appear in the image. That’s why a CNN with similar parameters can reach 93% where my MLP hits a wall at 51%. The architecture is the bottleneck, not the optimiser or the loss function.”

Interview Questions and Answers

Questions About Backpropagation

“Backpropagation is the algorithm for computing gradients in a neural network. During training, the network makes a prediction and we compute the loss. Then we work backwards through the network layer by layer using the chain rule: at each layer, we compute how much that layer’s weights contributed to the overall error, and we propagate the error signal one step further back. After one full backward pass, we have gradients for every weight in the network, which we use to update the weights.”

“The chain rule says: if $y = f(g(x))$, then $dy/dx = (dy/dg) \cdot (dg/dx)$. Neural networks are composed functions — each layer’s output is the input to the next layer. To find how the loss changes with respect to an early layer’s weights, you need to multiply the gradients through all the layers in between. That multiplication is what the chain rule enables and what backpropagation implements.”

“I implemented gradient checking. For each weight, I compute the gradient two ways: analytically via backprop, and numerically by slightly increasing and decreasing the weight by $\varepsilon = 10^{-5}$ and measuring the change in loss. The relative error between the two is about 10^{-10} , which is essentially machine precision. This confirms the math is correct.”

“When you have deep networks with sigmoid or tanh activations, the derivative of these functions is at most 0.25 (sigmoid) or 1 (tanh) — and much less when the inputs are large. As you multiply these derivatives together during backpropagation across many layers, the gradient gets exponentially smaller. By the time you reach the early layers, the gradient is nearly zero and those weights barely update. ReLU solves this because its derivative is 1 for all positive inputs.”

Questions About the Optimiser

“SGD requires you to carefully choose a single learning rate for all weights. Adam adapts the learning rate per parameter: weights with consistent gradients get larger steps, weights with noisy gradients get smaller steps. It also uses momentum — a running average of past gradients — which smooths out oscillations. In practice, Adam typically converges faster and is less sensitive to the initial learning rate choice.”

“Adam maintains running averages m_t and v_t initialised to zero. In the first few steps, these averages are heavily biased towards zero because they haven’t had time to accumulate history. Bias correction divides by $(1 - \beta^t)$, which is close to zero early in training, inflating the estimate back to a reasonable scale. Without it, the first few updates would be unreasonably small.”

Questions About the Results

“MNIST digits are greyscale, centred, and structurally simple — a ‘1’ is always a vertical stroke, a ‘0’ is always an oval. The discriminating features map well to pixel-level statistics. CIFAR-10 has colour, texture, pose variation, and background clutter. An airplane can appear at any angle, in any colour, overlapping clouds. These require understanding spatial relationships and hierarchical features, which MLPs cannot capture.”

“That’s the entire point of that experiment. The project is about understanding the fundamentals, and one of the most important fundamentals is that architecture matters. By deliberately using an MLP on CIFAR-10 and showing it hits a hard ceiling at 51%, I demonstrate exactly why CNNs were invented and what property of the architecture makes the difference. A result of 93% with a CNN would look better on paper but teach you nothing.”

“The right tool is a CNN. Instead of flattening the image, a CNN applies local filters (e.g., 3x3 kernels) that slide across the image, detecting features like edges regardless of where they appear. Adding convolutional layers followed by pooling, with batch normalisation and potentially residual connections (ResNet-style), would push accuracy to 90%+. But that’s a different architecture, not something this MLP framework can do.”

Questions About the Framework Design

“The key is storing weights and biases as Python lists: `self.weights[0]`, `self.weights[1]`, etc. The forward pass is a loop over `range(n_layers)`, applying each weight matrix and activation in sequence. Backprop is the same loop in reverse. Adding more layers is just a matter of passing a longer `layer_sizes` list — the code handles the rest automatically.”

“I maintain a list called `_cache` where `_cache[0]` is the raw input, `_cache[1]` is the output of the first layer, `_cache[2]` the second, and so on. During backpropagation, layer i needs the output of layer $i - 1$ to compute the gradient. Since we already computed all of these during the forward pass, we just look them up in the cache rather than recomputing.”

“Yes. L2 adds a penalty $(\lambda/2) \sum \|W\|^2$ to the loss, which encourages small weights and prevents the network from memorising the training data. In the gradient, it adds a term $(\lambda/n) \cdot W$ to each weight gradient, effectively “pulling” the weights towards zero at each step. In my implementation, you pass `12_lambda` to the constructor and it’s automatically included in the gradient computation.”

General ML Questions This Project Prepares You For

“Overfitting is when a model learns the training data too well — including its noise and peculiarities — and fails to generalise to new data. You detect it by tracking both training accuracy and test accuracy during training. If training accuracy keeps rising but test accuracy plateaus or falls, the model is overfitting. In this project, by epoch 30 the training accuracy is 99.6% and test accuracy is 97.7% — a 2% gap is normal and acceptable.”

“A one-hot vector represents a category as a vector of zeros with a single 1. For MNIST, digit 3 becomes `[0,0,0,1,0,0,0,0,0]`. We use it because the neural network’s output is a 10-dimensional probability vector, and we need the target (true label) in the same format to compute the loss. If we used the integer 3 directly, the loss function wouldn’t know how to compute the gradient correctly.”

“Two reasons. First, if features have very different scales (e.g., one pixel is in `[0,255]` and another is in `[0,1]`), the gradient landscape is poorly conditioned — the loss changes very quickly in some directions and slowly in others, making training slow. Second, typical weight initialisations assume inputs with mean 0 and variance 1. If inputs are in `[0,255]`, the pre-activations will be huge and we’ll spend epochs fighting exploding gradients before the network stabilises.”

“A mini-batch is a small subset of the training data processed together. Full-batch (all data at once) is expensive for large datasets and gives exactly the same gradient every step. Single-sample (stochastic) is noisy and doesn’t use vectorisation efficiently. A mini-batch of 256 is a middle ground: computationally efficient due to matrix operations, and the random sampling of each batch introduces beneficial noise that helps escape local minima.”

Key Numbers to Memorise

Fact	Value
MNIST test accuracy (this project)	97.66%
MNIST accuracy with old MSE loss	92%
CIFAR-10 MLP test accuracy	50.95%
CIFAR-10 MLP train accuracy	95.38% (overfits)
CNN accuracy on CIFAR-10 (reference)	~93%
Random guessing (10-class)	10%
MNIST: training samples	60,000
MNIST: test samples	10,000
MNIST: input size	784 (28×28)
MNIST: architecture	$784 - 128 - 10$
MNIST: parameters	101,770
CIFAR-10: training samples	50,000
CIFAR-10: input size	3,072 ($32 \times 32 \times 3$)
CIFAR-10: architecture	$3072 - 512 - 256 - 10$
Adam: β_1	0.9
Adam: β_2	0.999
Adam: ε	10^{-8}
Learning rate	0.001
Batch size	256 (MNIST), 512 (CIFAR-10)
Gradient check relative error	$\approx 10^{-10}$

What Makes This Project Stand Out

Here are the specific things that differentiate this project from “I trained a neural network” projects. Mention these in interviews:

1. **Mathematical verification:** You proved the backpropagation is correct to 10^{-10} precision using gradient checking. Most people never verify this.
2. **Explained a failure:** 51% on CIFAR-10 is not a bug. It’s a demonstrated, explained phenomenon with an architectural reason. Being able to explain *why* something fails shows deeper understanding than just tuning until something works.
3. **Concrete insight about loss functions:** You measured the difference between MSE and cross-entropy (92% vs 97.66%), not just heard that CE is better.
4. **Proper experimental design:** Architecture search with a held-out test set, multiple trials, mean and std reported. This is how real experiments are run.
5. **End-to-end:** From training, to saving/loading weights, to a working web demo. Shows you can connect all the pieces, not just the math.

What You Should NOT Claim

Be honest in interviews. These are the limits of this project:

- **Do not claim expertise in CNNs.** This project uses fully-connected networks only.
- **Do not claim expertise in PyTorch or TensorFlow.** This project deliberately avoids them.
- **Do not claim state-of-the-art results.** 97.66% on MNIST is good for a from-scratch MLP, not competitive with CNNs (99.7%).
- **Do not claim scalability.** A NumPy implementation is educational, not production-ready. Real frameworks use CUDA, automatic differentiation, and parallelism.

What you CAN claim: Deep understanding of how neural networks work at the mathematical level. The ability to implement, debug, verify, and explain each component from first principles. That is exactly what this project demonstrates.

Quick Revision — The 10-Point Cheat Sheet

Read this the morning before an interview:

1. **Project:** Mini deep learning framework in NumPy. N layers, ReLU, Adam, gradient check.
2. **Key result:** 97.66% on MNIST with 784–128–10 network. Was 92% with old code (MSE loss).
3. **Why the improvement?** Cross-entropy + Softmax gradient = $(\hat{y} - y)/n$. Clean signal. MSE gradient is weak near 0 and 1 — vanishing gradients.
4. **CIFAR-10 51%:** MLP cannot see spatial structure. CNNs use local filters. This is the demonstration, not a failure.
5. **Backprop:** Chain rule applied layer by layer, working backwards from the loss.
6. **Adam:** Momentum (smooth gradient average) + adaptive step size (divide by root of squared gradient average). Bias correction for first few steps.
7. **Gradient check:** Numerical verification. Perturb each weight $\pm\epsilon$, compare slope with analytical gradient. My error: 10^{-10} .
8. **He init for ReLU:** $\mathcal{N}(0, \sqrt{2/\text{fan_in}})$. Xavier for sigmoid/tanh: $\mathcal{N}(0, \sqrt{2/(\text{fan_in} + \text{fan_out})})$.
9. **L2 reg:** Penalise large weights. Adds $(\lambda/n)W$ to gradient.
10. **Architecture search:** Tested 6 hidden sizes, 2 trials each, always measured on held-out test set.

Built without frameworks — because understanding what happens inside `model.fit()` matters.